

POLITECHNIKA RZESZOWSKA

KATEDRA INFORMATYKI I AUTOMATYKI

DR INŻ. MATEUSZ POMIANEK

ĆWICZENIE LABORATORYJNE – Testy 1

GitHub · Testowanie · CI/CD z GitHub Actions

Programowanie Aplikacji Mobilnych (Android / Kotlin / Jetpack Compose)

Spis treści

1. Git i GitHub: podstawy profesjonalnego workflow	1
2. Tworzenie i konfiguracja publicznego repozytorium	2
3. Struktura projektu na Androida i konwencje kodu	4
4. Testy jednostkowe z JUnit 5 i MockK.....	4
5. Testowanie coroutines i Flow.....	6
6. Testy integracyjne: Room, Retrofit i Hilt	6
7. Testy interfejsu użytkownika z Espresso / Compose UI Test	7
8. Analiza statyczna: Detekt, ktlint i Android Lint.....	8
9. Pokrycie kodu: JaCoCo	8
10. GitHub Actions: kompletny potok CI/CD.....	9
11. Badges, szablony PR i ochrona gałęzi	10
12. Zadania do wykonania	11
13. Kryteria oceniania	12
14. Najczęstsze błędy i ich rozwiązania	12

1. Git i GitHub: podstawy profesjonalnego workflow

Git to rozproszony system kontroli wersji. GitHub to platforma hostująca repozytoria Git oraz dodająca narzędzia pracy zespołowej: pull requesty, recenzje kodu, issues, wiki i GitHub Actions. Rozróżnienie jest istotne: Git działa lokalnie bez GitHuba, natomiast GitHub jest usługą opartą na repozytoriach Git.

Analogia

Git jest dziennikiem pokładowym statku: zapisuje każdą zmianę kursu, autora i czas. GitHub jest portem macierzystym: przechowuje kopię dziennika, umożliwia wspólną pracę oraz organizuje odprawę jako-sciową przed wejściem kodu do gałęzi main.

1.1. Model gałęzi: GitHub Flow

W projektach mobilnych skuteczny jest prosty model: main zawsze gotowa do wydania, krótkotrwałe gałęzie funkcjonalne i obowiązkowy pull request przed scaleniem.

Gałąź	Cel	Kto tworzy / scala	Zasada
main	Kod stabilny. Kompiluje się i przechodzi testy.	Scalanie przez PR	Nigdy nie wykonuj bezpośredniego push do main.
feature/nazwa	Pojedyncza funkcja lub poprawka.	Deweloper	Nazwy: feature/weather-search, fix/crash-on-startup.
hotfix/nazwa	Krytyczna poprawka produkcyjna.	Senior / maintainer	Tylko dla błędów blokujących użytkownika.
develop	Opcjonalna gałąź integracyjna.	Zespół	Używana częściej w Gitflow niż w GitHub Flow.

1.2. Conventional Commits

Conventional Commits porządkują historię zmian i umożliwiają automatyczne generowanie CHANGELOG oraz wersjonowanie semantyczne.

```
# Format: <typ>(<zakres>): <opis>
# Typy: feat, fix, docs, style, refactor, test, chore, perf, ci

git commit -m "feat(weather): dodaj wyszukiwanie miast po nazwie"
git commit -m "fix(api): napraw parsowanie temperatury ujemnej"
git commit -m "test(viewmodel): dodaj testy jednostkowe HomeViewModel"
git commit -m "ci: skonfiguruj GitHub Actions z JUnit i JaCoCo"

git commit -m "feat(api)!: zmień format odpowiedzi WeatherApi"
# Wykrzyknik oznacza breaking change.
```

Automatyczna weryfikacja

Workflow może odrzucać PR z niepoprawnym tytułem albo commitami typu „fix2”, „poprawka”, „aaa”. Dzięki temu historia repozytorium pozostaje czytelna przez cały semestr.

2. Tworzenie i konfiguracja publicznego repozytorium**2.1. Inicjalizacja repozytorium**

```
# GitHub > New repository
# Nazwa: WeatherApp-Android
# Widoczność: Public
# .gitignore: Android
# Licencja: Apache 2.0

git clone https://github.com/TWOJ_LOGIN/WeatherApp-Android.git
cd WeatherApp-Android

git config --global user.name "Jan Kowalski"
git config --global user.email "jan.kowalski@stud.prz.edu.pl"
```

2.2. .gitignore dla Androida

Poprawny .gitignore zapobiega dodaniu plików generowanych, lokalnych ścieżek SDK i sekretów. Szczególnie istotny jest local.properties oraz pliki keystore.

```
*.iml
.gradle/
/local.properties
/.idea/
/*build/

*.jks
*.keystore
key.properties
google-services.json
local.defaults.properties

*.apk
*.aab
*.ap_
```

2.3. README.md

README powinien pozwolić nowej osobie uruchomić projekt i testy w krótkim czasie. Minimalny zakres to: opis, wymagania, uruchomienie, testy, architektura i status CI.

```
# WeatherApp

[![CI](https://github.com/LOGIN/WeatherApp-Android/actions/workflows/ci.yml/badge.svg)](...)
[![Coverage](https://codecov.io/gh/LOGIN/WeatherApp-Android/branch/main/graph/badge.svg)](...)]

Aplikacja pogodowa pobierająca dane z Open-Meteo API.

## Wymagania
- Android Studio
- JDK 17
- Android SDK API 34+

## Testy
```bash
./gradlew test
./gradlew connectedAndroidTest
./gradlew detekt
./gradlew jacocoTestReport
```

## 2.4. Szablony issues i pull requestów

```
mkdir -p .github/ISSUE_TEMPLATE
.github/ISSUE_TEMPLATE/bug_report.md

name: Zgłoszenie błędu
about: Poinformuj o problemie, aby pomóc nam go naprawić
labels: bug

Opis błędu
Kroki do odtworzenia
Oczekiwane zachowanie
Aktualne zachowanie
Środowisko

.github/pull_request_template.md
Opis zmian
Powiązany issue
Closes #
```

```
Checklist
- [] Kod kompiluje się bez ostrzeżeń
- [] Dodano lub zaktualizowano testy
- [] ./gradlew test przechodzi lokalnie
- [] Detekt i ktlint nie zgłaszają nowych problemów
```

### 3. Struktura projektu na Androida i konwencje kodu

Projekt WeatherApp stosuje MVVM z elementami Clean Architecture. Kierunek zależności jest jednokierunkowy: UI zna Domain, Domain nie zna UI ani Data, Data implementuje interfejsy zdefiniowane w Domain. Taki podział upraszcza testowanie i wymianę implementacji.

```
app/
├── src/main/kotlin/pl/edu/pam/weather/
│ ├── ui/
│ │ ├── screens/home/HomeScreen.kt
│ │ ├── screens/home/HomeViewModel.kt
│ │ └── components/WeatherCard.kt
│ ├── domain/
│ │ ├── model/WeatherData.kt
│ │ ├── repository/WeatherRepository.kt
│ │ └── usecase/GetWeatherUseCase.kt
│ └── data/
│ ├── remote/WeatherApi.kt
│ ├── remote/dto/WeatherDto.kt
│ ├── local/WeatherDatabase.kt
│ └── WeatherRepositoryImpl.kt
├── di/AppModule.kt
├── src/test/kotlin/... # testy JVM
└── src/androidTest/kotlin/... # testy instrumentowane
```

#### 3.1. Zależności testowe

```
dependencies {
 testImplementation("org.junit.jupiter:junit-jupiter:5.10.2")
 testImplementation("io.mockk:mockk:1.13.10")
 testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.8.0")
 testImplementation("app.cash.turbine:turbine:1.1.0")
 testImplementation("com.google.truth:truth:1.4.2")

 androidTestImplementation("androidx.test.ext:junit:1.1.5")
 androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
 androidTestImplementation("androidx.compose.ui:ui-test-junit4")
 androidTestImplementation("com.google.dagger:hilt-android-testing:2.51")
 androidTestImplementation("androidx.room:room-testing:2.6.1")
}

tasks.withType<Test> {
 useJUnitPlatform()
}
```

### 4. Testy jednostkowe z JUnit 5 i MockK

Test jednostkowy sprawdza pojedynczą klasę lub funkcję w izolacji od zależności. W projekcie Android typowo testuje się use case, ViewModel, mapery DTO oraz walidację danych.

#### Analogia

Test jednostkowy przypomina test części samochodu poza samochodem. Silnik, hamulce i skrzynia biegów można sprawdzić osobno, zanim złożymy cały system w testach integracyjnych.

Element	Znaczenie
@ExtendWith(MockKExtension::class)	Integracja MockK z JUnit 5.

@MockK	Mock wymagający jawnego zdefiniowania zachowania.
@RelaxedMockK	Mock zwracający wartości domyślne.
coEvery { ... } returns ...	Definicja zachowania funkcji suspend.
coVerify { ... }	Weryfikacja wywołania funkcji suspend.
assertThat(...).isEqualTo(...)	Czytelne asercje Google Truth.

## 4.1. Przykład: GetWeatherUseCaseTest

```
@ExtendWith(MockKExtension::class)
class GetWeatherUseCaseTest {

 @MockK
 private lateinit var repository: WeatherRepository
 private lateinit var useCase: GetWeatherUseCase

 @BeforeEach
 fun setUp() {
 useCase = GetWeatherUseCase(repository)
 }

 @Test
 fun `returns weather data for valid city`() = runTest {
 val expected = WeatherData(
 city = "Warszawa",
 temperature = 22.5,
 humidity = 65,
 windSpeed = 12.0,
 weatherCode = 1,
)
 coEvery { repository.getWeather("Warszawa") } returns Result.success(expected)
 val result = useCase("Warszawa")
 assertThat(result.isSuccess).isTrue()
 assertThat(result.getOrNull()).isEqualTo(expected)
 coVerify(exactly = 1) { repository.getWeather("Warszawa") }
 }

 @Test
 fun `returns failure when repository throws exception`() = runTest {
 coEvery { repository.getWeather(any()) } throws IOException("Brak sieci")
 val result = useCase("Gdańsk")
 assertThat(result.isFailure).isTrue()
 assertThat(result.exceptionOrNull()).isInstanceOf(IOException::class.java)
 }
}
```

## 4.2. Test mapowania DTO

```
@Test
fun `maps dto to domain model correctly`() {
 val dto = WeatherDto(
 currentWeather = CurrentWeatherDto(
 temperature = 22.5,
 windspeed = 12.0,
 weathercode = 1,
)
)
 val result = dto.toDomain(cityName = "Warszawa")
 assertThat(result.city).isEqualTo("Warszawa")
 assertThat(result.temperature).isWithin(0.001).of(22.5)
 assertThat(result.windSpeed).isWithin(0.001).of(12.0)
 assertThat(result.weatherCode).isEqualTo(1)
}
```

## 5. Testowanie coroutines i Flow

Kod asynchroniczny wymaga kontroli wirtualnego czasu oraz obserwacji emisji. `kotlinx-coroutines-test` dostarcza `runTest`, `StandardTestDispatcher` i `advanceUntilIdle`, a `Turbine` upraszcza testowanie `Flow`.

```
@Test
fun `uiState shows loading then success after search`() = runTest {
 val weatherData = WeatherData(city = "Poznań", temperature = 18.0)
 coEvery { getWeather("Poznań") } returns Result.success(weatherData)

 viewModel.searchCity("Poznań")

 assertThat(viewModel.uiState.value).isInstanceOf(UiState.Loading::class.java)
 advanceUntilIdle()

 val state = viewModel.uiState.value
 assertThat(state).isInstanceOf(UiState.Success::class.java)
 assertThat((state as UiState.Success).data.city).isEqualTo("Poznań")
}
```

### 5.1. Turbine

```
@Test
fun `favorites flow emits updated list after add`() = runTest {
 val repository = FakeWeatherRepository()

 repository.getFavorites().test {
 assertThat(awaitItem()).isEmpty()

 repository.addFavorite("Łódź")

 val updated = awaitItem()
 assertThat(updated).hasSize(1)
 assertThat(updated.first().city).isEqualTo("Łódź")

 cancelAndIgnoreRemainingEvents()
 }
}
```

Obiekt testowy	Opis	Kiedy stosować
Mock	Obiekt generowany przez bibliotekę, konfigurowany przez <code>every/coEvery</code> .	Gdy weryfikujesz interakcje.
Stub	Zależność zwracająca proste, z góry ustalone odpowiedzi.	Gdy nie interesuje Cię liczba wywołań.
Fake	Ręczna, uproszczona implementacja interfejsu.	Gdy potrzebny jest stan i bardziej realistyczne zachowanie.
Spy	Prawdziwy obiekt częściowo śledzony lub podmieniany.	Rzadko; zwykle przy testach kodu legacy.

## 6. Testy integracyjne: Room, Retrofit i Hilt

Testy integracyjne sprawdzają współpracę komponentów: DAO z bazą w pamięci, Retrofit z lokalnym serwerem HTTP oraz Hilt z testowymi modułami zależności.

### 6.1. Room DAO

```
class WeatherDaoTest {
 private lateinit var database: WeatherDatabase
 private lateinit var dao: WeatherDao
}
```

```

@BeforeEach
fun setUp() {
 val context = ApplicationProvider.getApplicationContext<Context>()
 database = Room.inMemoryDatabaseBuilder(context, WeatherDatabase::class.java)
 .allowMainThreadQueries()
 .build()
 dao = database.weatherDao()
}

@AfterEach
fun tearDown() {
 database.close()
}

@Test
fun `insert and retrieve weather entity`() = runTest {
 val entity = WeatherEntity(city = "Białystok", temperature = 15.2)
 dao.insertWeather(entity)
 val result = dao.getWeatherForCity("Białystok")
 assertThat(result).isNotNull()
 assertThat(result!!.temperature).isWithin(0.001).of(15.2)
}
}

```

## 6.2. Retrofit + MockWebServer

```

@Test
fun `parses successful weather response correctly`() = runTest {
 mockWebServer.enqueue(
 MockResponse()
 .setResponseCode(200)
 .setBody("""
 { "current": {
 "temperature_2m": 22.5,
 "relative_humidity_2m": 65,
 "wind_speed_10m": 12.0,
 "weather_code": 1
 } }
 """).trimIndent())
)
 val result = api.getCurrentWeather(52.23, 21.01, "temperature_2m")
 assertThat(result.current.temperature2m).isWithin(0.001).of(22.5)
 assertThat(result.current.relativeHumidity2m).isEqualTo(65)
}

```

## 7. Testy interfejsu użytkownika z Espresso / Compose UI Test

Testy UI uruchamiają aplikację na emulatorze lub urządzeniu. Są najdroższe w utrzymaniu, dlatego powinny sprawdzać krytyczne przepływy użytkownika, a nie każdą drobną gałąź logiki.

```

@HiltAndroidTest
@RunWith(AndroidJUnit4::class)
class HomeScreenTest {

 @get:Rule(order = 0)
 val hiltRule = HiltAndroidRule(this)

 @get:Rule(order = 1)
 val composeRule = createAndroidComposeRule<MainActivity>()

 @Before
 fun setUp() {
 hiltRule.inject()
 }

 @Test
 fun searchCity_displaysWeatherCard() {
 with(composeRule) {
 onNodeWithText("Wpisz nazwę miasta").performClick()
 onNodeWithText("Wpisz nazwę miasta").performTextInput("Rzeszów")
 }
 }
}

```

```

 onNodeWithContentDescription("Szukaj").performClick()
 onNodeWithText("Rzeszów").assertIsDisplayed()
 onNodeWithText("20.5°C").assertIsDisplayed()
 }
}
./gradlew connectedAndroidTest
./gradlew connectedAndroidTest \
-Pandroid.testInstrumentationRunnerArguments.class=pl.edu.pam.weather.ui.HomeScreenTest

```

## 8. Analiza statyczna: Detekt, ktlint i Android Lint

Analiza statyczna sprawdza kod bez jego uruchamiania. Detekt wykrywa problemy jakościowe, ktlint pilnuje formatowania, a Android Lint sprawdza problemy specyficzne dla platformy Android.

Narzędzie	Co sprawdza	Kiedy uruchamiać
Detekt	Złożoność, długie metody, duplikację, magiczne liczby, martwy kod.	Przed commitem i w CI.
ktlint	Formatowanie, importy, wcięcia, styl Kotlin.	Lokalnie przez ktlintFormat; w CI przez ktlintCheck.
Android Lint	Uprawnienia, zasoby, przestarzałe API, problemy manifestu.	W Android Studio i w CI.

```

detekt {
 config.setFrom("$rootDir/config/detekt/detekt.yml")
 buildUponDefaultConfig = true
 autoCorrect = false
}

config/detekt/detekt.yml
complexity:
 LongMethod:
 threshold: 60
 CyclomaticComplexMethod:
 threshold: 15
style:
 MaxLineLength:
 maxLineLength: 120

```

### 8.1. Pre-commit hook

```

#!/bin/sh
set -e

echo "Uruchamianie Detekt i ktlintCheck..."
./gradlew detektMain
./gradlew ktlintCheck

echo "Weryfikacja przeszła pomyślnie."

```

## 9. Pokrycie kodu: JaCoCo

JaCoCo mierzy, które instrukcje, linie, metody i gałęzie zostały wykonane przez testy. Pokrycie jest wskaźnikiem pomocniczym: wysokie pokrycie nie gwarantuje jakości testów, ale niskie pokrycie wskazuje obszary ryzyka.

```

tasks.register<JacocoReport>("jacocoTestReport") {
 dependsOn("testDebugUnitTest")

 reports {
 xml.required.set(true)
 html.required.set(true)
 csv.required.set(false)
 }
}

```



```

val excludes = listOf(
 "**/di/**",
 "**/*_Factory*",
 "**/*_HiltModules*",
 "**/R.class",
 "**/R$.class",
 "**/BuildConfig.*",
)

sourceDirectories.setFrom(files("src/main/kotlin", "src/main/java"))
executionData.setFrom(fileTree(layout.buildDirectory) {
 include("**/*.exec", "**/*.ec")
})
}

```

Metryka	Co mierzy	Cel
Instruction Coverage	Instrukcje bajtkodu wykonane przez testy.	> 70% dla domain/usecase.
Branch Coverage	Gałęzie if/else/when wykonane przez testy.	> 60% dla logiki warunkowej.
Line Coverage	Linie kodu uruchomione w testach.	> 70% dla klas domenowych.
Method Coverage	Metody wywołane przez testy.	> 80% dla logiki biznesowej.

## 10. GitHub Actions: kompletny potok CI/CD

GitHub Actions automatyzuje kontrolę jakości: analizę statyczną, testy jednostkowe, raport pokrycia oraz testy instrumentowane. Workflow powinien działać dla push i pull\_request.

name: **CI** - Testy i analiza statyczna

```

on:
 push:
 branches: [main, develop]
 pull_request:
 branches: [main]
 workflow_dispatch:

concurrency:
 group: ${{ github.workflow }}-${{ github.ref }}
 cancel-in-progress: true

env:
 JAVA_VERSION: '17'

jobs:
 static-analysis:
 name: Detekt + ktlint + Android Lint
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-java@v4
 with:
 java-version: ${{ env.JAVA_VERSION }}
 distribution: temurin
 cache: gradle
 - run: chmod +x gradlew
 - run: ./gradlew detektMain detektTest --continue
 - run: ./gradlew ktlintCheck
 - run: ./gradlew lint

 unit-tests:
 name: Testy jednostkowe + JaCoCo
 runs-on: ubuntu-latest
 needs: static-analysis
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-java@v4
 with:

```

```

 java-version: ${ env.JAVA_VERSION }}
 distribution: temurin
 cache: gradle
 - run: chmod +x gradlew
 - run: ./gradlew testDebugUnitTest --continue
 - run: ./gradlew jacocoTestReport
 - uses: codecov/codecov-action@v4
 with:
 token: ${ secrets.CODECOV_TOKEN }}
 files: app/build/reports/jacoco/jacocoTestReport/jacocoTestReport.xml
 fail_ci_if_error: true

instrumented-tests:
 name: Testy UI na emulatorze
 runs-on: ubuntu-latest
 needs: unit-tests
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-java@v4
 with:
 java-version: ${ env.JAVA_VERSION }}
 distribution: temurin
 cache: gradle
 - run: chmod +x gradlew
 - uses: reactivecircus/android-emulator-runner@v2
 with:
 api-level: 34
 arch: x86_64
 disable-animations: true
 script: ./gradlew connectedDebugAndroidTest --stacktrace

```

## 10.1. Walidacja pull requesta

```

name: Walidacja Pull Requesta

on:
 pull_request:
 types: [opened, edited, synchronize]

jobs:
 validate-pr-title:
 runs-on: ubuntu-latest
 steps:
 - uses: amannn/action-semantic-pull-request@v5
 env:
 GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }}
 with:
 types: |
 feat
 fix
 docs
 style
 refactor
 test
 chore
 ci
 perf
 requireScope: false

```

## 11. Badges, szablony PR i ochrona gałęzi

Badges są dynamicznymi elementami README pokazującymi stan projektu. Ochrona gałęzi main wymusza proces code review i przejście testów przed scaleniem.

```

[![CI](https://github.com/USER/REPO/actions/workflows/ci.yml/badge.svg?branch=main)](...)
[![Coverage](https://codecov.io/gh/USER/REPO/graph/badge.svg)](...)
[![License: Apache 2.0](https://img.shields.io/badge/License-Apache%202.0-blue.svg)](...)
[![Kotlin](https://img.shields.io/badge/Kotlin-2.0-7F52FF.svg?logo=kotlin)](...)

```

```
[![Min SDK](https://img.shields.io/badge/Android-API%2026%2B-brightgreen?logo=android)](...)
```

Ustawienie	Co robi
Require a pull request before merging	Blokuje bezpośredni push do main.
Require approvals: 1	Wymaga co najmniej jednej akceptacji.
Require status checks to pass	Wymaga zielonych jobów CI.
Require branches to be up to date	Wymaga aktualizacji gałęzi PR z main.
Require conversation resolution	Wymaga rozwiązania komentarzy w PR.

```
.github/CODEOWNERS
app/src/main/kotlin/ @prowadzacy-login
.github/workflows/ @prowadzacy-login @student-login
* @prowadzacy-login
```

## 12. Zadania do wykonania

Zadania budują kompletne repozytorium projektu WeatherApp w Kotlinie z testami oraz automatyzacją CI/CD. Realizuj je w podanej kolejności.

### Weryfikacja

Przedstaw repozytorium, wynik komend Gradle, screenshot zielonego workflow oraz raport pokrycia.

### Zadanie 1: Repozytorium i konwencje

1. Załóż publiczne repozytorium WeatherApp-Android z licencją Apache 2.0 i szablonem .gitignore dla Androida.
2. Skonfiguruj .gitignore tak, aby ignorował local.properties, pliki keystore, build/ i artefakty APK/AAB.
3. Przygotuj README.md z opisem, wymaganiami, instrukcją uruchomienia, testami i miejscem na badges.
4. Dodaj szablony issue i pull requesta.
5. Włącz Branch Protection dla main: PR, jedna akceptacja i status checks.

### Zadanie 2: Testy na wszystkich poziomach

1. Dodaj zależności testowe: JUnit 5, MockK, coroutines-test, Turbine i Truth.
2. Napisz minimum 8 testów jednostkowych dla use case, mapperów i logiki ulubionych miast.
3. Napisz minimum 3 testy coroutines / Flow z runTest i Turbine.
4. Napisz minimum 3 testy integracyjne Room DAO.
5. Napisz minimum 4 testy UI na emulatorze z testowymi zależnościami Hilt.

### Zadanie 3: GitHub Actions CI/CD

1. Utwórz ci.yml z jobami static-analysis, unit-tests i instrumented-tests.
2. Skonfiguruj Detekt, ktlint i Android Lint.
3. Skonfiguruj JaCoCo i wysyłkę raportu XML do Codecov.
4. Dodaj workflow walidacji PR z Conventional Commits.
5. Zaktualizuj README o faktyczne badges oraz liczbę testów.

### Zadanie 4: Jakość kodu i dokumentacja

1. Skonfiguruj pre-commit hook uruchamiający Detekt i ktlint.
2. Dodaj CHANGELOG.md zgodny z Keep a Changelog.
3. Skonfiguruj Dependabot dla Gradle i GitHub Actions.
4. Na podstawie raportu JaCoCo popraw pokrycie co najmniej jednej klasy poniżej 50%.

## 13. Kryteria oceniania

Zadanie	Punkty	Co weryfikuje prowadzący
Repozytorium	20	Publiczne repo, README, szablony, Branch Protection, pierwszy PR.
Testy	30	Minimum 14 testów JVM i 4 testy UI. Wszystkie przechodzą.
CI/CD	30	Trzy zielone joby, raport JaCoCo, walidacja PR, badges.
Jakość i dokumentacja	20	Pre-commit hook, CHANGELOG, Dependabot i poprawa pokrycia.
Razem	100	Ocena końcowa zgodnie z progami.

Ocena	Punkty / wymagania
5.0	90–100 pkt; wszystkie zadania, pokrycie > 75%, min. 25 testów.
4.5	80–89 pkt; zadania 1–3 kompletne, pokrycie > 65%, min. 20 testów.
4.0	70–79 pkt; repozytorium, testy i podstawowy CI.
3.5	60–69 pkt; zadania 1–2 i częściowy CI.
3.0	50–59 pkt; repozytorium, minimum 6 testów JVM i podstawowy workflow.
2.0	0–49 pkt; brak repozytorium, zbyt mało testów lub CI nie uruchamia się.

## 14. Najczęstsze błędy i ich rozwiązania

Błąd / objaw	Przyczyna i rozwiązanie
local.properties trafił do repozytorium	Usuń z indeksu: <code>git rm --cached local.properties</code> , dodaj wpis do <code>.gitignore</code> i wykonaj commit.
No tests found for given includes	Brakuje <code>useJUnitPlatform()</code> albo importujesz <code>@Test</code> z JUnit 4 zamiast JUnit 5.
coEvery unresolved reference	Brakuje importu <code>io.mockk.coEvery</code> albo funkcja nie jest suspend.
Android Gradle plugin requires Java 17	Ustaw <code>java-version: 17</code> w <code>setup-java</code> i sprawdź lokalnie <code>./gradlew --version</code> .
Raport JaCoCo ma 0%	Sprawdź <code>dependsOn(testDebugUnitTest)</code> , <code>executionData</code> i katalogi z klasami Kotlin.
Detekt blokuje MagicNumber	Wyodrębnij stałe albo dopisz uzasadnione wartości do <code>ignoreNumbers</code> .
Test UI nie widzi elementu	Sprawdź wstrzykiwanie Hilt, stabilność identyfikatorów semantics i oczekiwanie na stan UI.
Permission denied przy <code>./gradlew</code>	Dodaj <code>chmod +x gradlew</code> w workflow albo zatwierdź uprawnienie wykonania w repozytorium.
Codecov badge pokazuje unknown	Sprawdź <code>CODECOV_TOKEN</code> i ścieżkę do raportu XML.
Turbine nie otrzymuje elementu	Flow nie emituje wartości; zweryfikuj kolejność akcji i użycie <code>runTest</code> .

## Przydatne komendy Gradle

```
./gradlew test
./gradlew testDebugUnitTest
./gradlew test --tests 'pl.edu.pam.weather.usecase.GetWeatherUseCaseTest'
./gradlew connectedAndroidTest
./gradlew detektMain
./gradlew ktlintCheck
./gradlew ktlintFormat
./gradlew lint
./gradlew jacocoTestReport
./gradlew clean
```

**Podsumowanie:** ćwiczenie koncentruje się na inżynierii oprogramowania: poprawnej historii repozytorium, testowalnej architekturze, automatyzacji kontroli jakości i przejrzystej dokumentacji.

## ĆWICZENIE LABORATORYJNE – Testy 2

### GitHub · Testowanie · CI/CD z GitHub Actions

Programowanie Aplikacji Mobilnych (Android / Kotlin / Jetpack Compose)

#### Spis treści

15. Zaawansowane mockowanie z MockK.....	13
16. Budowanie FakeRepository krok po kroku .....	16
17. Testowanie kompletnego stosu MVVM .....	20
18. Snapshot testing widoków Compose z Paparazzi .....	25
19. Testy wydajności: Android Macrobenchmark .....	28
20. Zaawansowany GitHub Actions: macierze i artefakty .....	30
21. Code review w praktyce: narzędzia i workflow .....	36
22. Bezpieczeństwo repozytorium: skanowanie zależności .....	38
23. Kompletny sprint: od Issue do scalenia z main.....	40

#### 15. Zaawansowane mockowanie z MockK

W ćwiczeniu 1 z testów poznałeś podstawy MockK: `@MockK`, `coEvery` i `coVerify`. To ćwiczenie pokazuje techniki potrzebne w rzeczywistych projektach: przechwytywanie argumentów (`capture`), szpiegowanie prawdziwych obiektów (`spy`), odpowiedzi zależne od argumentu (`answers`) i sprawdzanie kolejności wywołań.

##### SPY: testowanie cz. prawdziwej implementacji

`Spy` opakowuje prawdziwy obiekt. Metody, które nie są podmienione, działają normalnie. Podmienione zachowują się jak `mock`. Używaj `spy` gdy chcesz sprawdzić, czy prawdziwa metoda wywołuje inne metody w odpowiedniej kolejności.

```
// Scenariusz: WeatherRepositoryImpl ma metode getWeather(), która wewnętrznie
// wywołuje _fetchFromNetwork() i _saveToCache(). Chcemy sprawdzić integrację.

class WeatherRepositoryImplTest {
```

```
// spyk() tworzy spy na prawdziwym obiekcie:
private val fakeApi = mockk<WeatherApi>(relaxed = true)
private val fakeDao = mockk<WeatherDao>(relaxed = true)
private val repository = spyk(WeatherRepositoryImpl(fakeApi, fakeDao))
// ^^^^
// spyk() = prawdziwa implementacja, ale możemy podmienić lub śledzić metody

@Test
fun `getWeather calls network and then saves to cache`() = runTest {
 val weatherData = buildWeatherData(city = "Wroclaw")

 // Podmień TYLKO metodę sieciową - reszta działa naprawdę:
 coEvery { repository.fetchFromNetwork("Wroclaw") } returns weatherData

 // Wywołaj prawdziwą getWeather():
 repository.getWeather("Wroclaw")

 // Zweryfikuj że oba kroki wystąpiły, w tej kolejności:
 coVerifyOrder {
 repository.fetchFromNetwork("Wroclaw") // 1. Najpierw sieć
 fakeDao.insertWeather(any()) // 2. Potem zapis do cache
 }
}
}
```

**Krok 1:** Dodaj test do klasy WeatherRepositoryImplTest w swoim projekcie.

**Krok 2:** Uruchom test: `./gradlew test --tests '*WeatherRepositoryImplTest'`

**Krok 3:** Dodaj deliberatnie błędną kolejność (najpierw save, potem fetch) i obserwuj komunikat błędu Mockk.

✓ **Weryfikacja:** Test przechodzi. Po zamianie kolejności w weryfikacji MockK wyświetla komunikat 'Verification failed: calls are not in order'.

✗ **Częsty błąd:** spyk() na klasach final (domyślnie w Kotlin) rzuca błąd. Dodaj open lub dodaj do build.gradle.kts: testImplementation('io.mockk:mockk-android:1.13.10') i configure dla 'all-open' plugin.

## CAPTURE: przechwytywanie argumentów

Capture pozwala przechwycić argument przekazany do mocka i sprawdzić jego wartość po wywołaniu. Jest przydatny gdy argument jest złożonym obiektem, który jest tworzony wewnątrz testowanej metody i nie jest bezpośrednio dostępny z zewnątrz.

```
// Scenariusz: chcemy sprawdzić dokładną treść obiektu WeatherEntity,
// który ViewModel przekazuje do DAO - bez dostępu do jego konstruktora.

class WeatherViewModelTest {
```

```
@MockK private lateinit var dao: WeatherDao
private val slot = slot<WeatherEntity>() // Pojemnik na przechwycony argument

@Test
fun `saved entity has correct city and timestamp`() = runTest {
 // Konfiguruj mock z capture:
 coEvery { dao.insertWeather(capture(slot)) } just Runs
 //
 // capture(slot) = zapisz argument do 'slot' przy wywołaniu

 val viewModel = HomeViewModel(GetWeatherUseCase(repository))
 viewModel.searchAndSave("Opole")
 advanceUntilIdle()

 // Teraz możemy sprawdzić co zostało przekazane do DAO:
 val capturedEntity = slot.captured
 assertThat(capturedEntity.city).isEqualTo("Opole")
 assertThat(capturedEntity.cachedAt).isGreaterThan(0L)
 assertThat(capturedEntity.temperature).isNotNull()
}

// Przechwytywanie listy argumentów (wiele wywołań):
val entitySlots = mutableListOf<WeatherEntity>()
coEvery { dao.insertWeather(capture(entitySlots)) } just Runs
// Po teście: entitySlots[0], entitySlots[1] itd.
```

**Krok 1:** Napisz test z capture dla swojego DAO: sprawdź że po wywołaniu save() do bazy trafi obiekt z poprawną nazwą miasta i że pole cachedAt jest pozytywną liczbą (timestamp > 0).

**Krok 2:** Uruchom test i sprawdź wynik.

**Krok 3:** Dodaj asercje sprawdzające że temperatura w przechwyconym obiekcie pochodzi z danych pogodowych, a nie jest zerowa.

✓ **Weryfikacja:** Wszystkie asercje na przechwyconym obiekcie przechodzą. Capture umożliwia sprawdzenie wewnętrznej logiki tworzenia obiektów.

## ANSWER: odpowiedź zależna od argumentu

Answers umożliwiają definiowanie odpowiedzi mocked metody jako funkcji argumentu - zamiast zawsze tej samej wartości. Przydatne gdy mock musi zachowywać się inaczej dla różnych danych wejściowych bez pisania wielu coEvery.

```
// Scenariusz: Repository ma zwracać różne dane dla różnych miast
// bez pisania osobnego coEvery dla każdego miasta.

@MockK private lateinit var repository: WeatherRepository
```

```
@BeforeEach
fun setUp() {
 // answers { } = lambda z dostepem do argumentow wywolania
 coEvery { repository.getWeather(any()) } answers {
 val city = firstArg<String>() // Pobierz pierwszy argument wywolania
 when (city) {
 "Slupsk" -> Result.success(buildWeatherData(city, temp = 14.0))
 "Kielce" -> Result.success(buildWeatherData(city, temp = 19.0))
 "Olsztyn" -> Result.failure(Exception("Miasto niedostepne"))
 else -> Result.success(buildWeatherData(city, temp = 20.0))
 }
 }
}

@Test
fun `answer lambda returns city-specific data`() = runTest {
 val slupsk = repository.getWeather("Slupsk")
 val kielce = repository.getWeather("Kielce")
 val olsztyn = repository.getWeather("Olsztyn")

 assertThat(slupsk.getOrNull()?.temperature).isWithin(0.01).of(14.0)
 assertThat(kielce.getOrNull()?.temperature).isWithin(0.01).of(19.0)
 assertThat(olsztyn.isFailure).isTrue()
}
```

**Krok 1:** Wdróż wzorec answers w swoich testach: jeden coEvery obsługujący wszystkie przypadki przez when (city).

**Krok 2:** Uruchom testy. Porównaj czytelność z podejściem wielu oddzielnych coEvery.

✓ **Weryfikacja:** Jeden blok coEvery answers zastępuje 3+ oddzielne bloki coEvery. Kod testowy jest krótszy i łatwy do rozszerzenia.

## 16. Budowanie FakeRepository krok po kroku

FakeRepository to ręczna implementacja interfejsu repozytorium, która przechowuje dane w pamięci. Jest bardziej realistyczna niż mock (zachowuje stan między wywołaniami), ale szybsza i bardziej kontrolowalna niż prawdziwa baza danych. To ćwiczenie buduje kompletny FakeWeatherRepository od zera, krok po kroku.

### Kiedy Fake jest lepszy od Mock?

Mock jest idealny gdy testujemy pojedyncze interakcje: 'czy metoda została wywołana z tym argumentem?'. Fake jest lepszy gdy testujemy zachowanie systemu w czasie: 'czy po dodaniu miasta do ulubionych i przeładowaniu aplikacji miasto jest nadal na liście?'. Mock statecznie odpowiada na pytania. Fake naprawdę przechowuje i zwraca dane.

## Stworzenie interfejsu do implementacji

```
// Jesli jeszcze nie masz interfejsu - stwórz go teraz.
```



```
// domain/repository/WeatherRepository.kt
interface WeatherRepository {
 suspend fun getWeather(city: String): Result<WeatherData>
 fun getFavorites(): Flow<List<WeatherData>>
 suspend fun addFavorite(city: String)
 suspend fun removeFavorite(city: String)
 suspend fun clearCache()
 suspend fun isFavorite(city: String): Boolean
}
```

**Krok 1:** Jeśli twój interfejs ma inne metody, dopasuj go. Kluczowe: `getFavorites()` musi zwracać `Flow` (nie `suspend`), aby Turbine mogło go testować.

## Implementacja FakeWeatherRepository

```
// test/kotlin/.../fake/FakeWeatherRepository.kt
// Konwencja: klasy Fake sa w katalogu fake/ wewnatrz katalogu testow
class FakeWeatherRepository : WeatherRepository {

 // Stan wewnetrzny - przechowuje dane jak prawdziwe zrodlo
 private val _weatherMap = mutableMapOf<String, WeatherData>()
 private val _favorites = MutableStateFlow<List<WeatherData>>(emptyList())
 private val _shouldFail = mutableMapOf<String, Exception>()
 private var _fetchCount = 0 // Licznik wywołan - do weryfikacji w testach

 // — API testowe (nie w interfejsie) —————

 /** Przygotuj dane do zwrocenia dla danego miasta. */
 fun givenWeatherFor(city: String, data: WeatherData) {
 _weatherMap[city] = data
 }

 /** Skonfiguruj blad dla danego miasta. */
 fun givenErrorFor(city: String, exception: Exception = Exception("Blad sieci")) {
 _shouldFail[city] = exception
 }

 /** Ile razy wywołano getWeather() - do weryfikacji w testach. */
 val fetchCount: Int get() = _fetchCount

 /** Resetuj stan do wartosci początkowych. */
 fun reset() {
 _weatherMap.clear()
 _favorites.value = emptyList()
 _shouldFail.clear()
 _fetchCount = 0
 }

 // — Implementacja interfejsu —————

 override suspend fun getWeather(city: String): Result<WeatherData> {
 _fetchCount++
 _shouldFail[city]?.let { return Result.failure(it) }
 return _weatherMap[city]
 }
}
```

```

 ?.let { Result.success(it) }
 ?: Result.failure(Exception("Brak danych dla: $city"))
 }

 override fun getFavorites(): Flow<List<WeatherData>> = _favorites.asStateFlow()

 override suspend fun addFavorite(city: String) {
 val data = _weatherMap[city] ?: return // Ignoruj jesli brak danych
 val current = _favorites.value
 if (current.none { it.city == city }) {
 _favorites.value = current + data // Niemutowalna aktualizacja
 }
 }

 override suspend fun removeFavorite(city: String) {
 _favorites.value = _favorites.value.filter { it.city != city }
 }

 override suspend fun clearCache() {
 _weatherMap.clear()
 _favorites.value = emptyList()
 _fetchCount = 0
 }

 override suspend fun isFavorite(city: String): Boolean {
 return _favorites.value.any { it.city == city }
 }
}

```

**Krok 1:** Stwórz plik FakeWeatherRepository.kt w katalogu test/kotlin/.../fake/.

**Krok 2:** Implementuj każdą metodę interfejsu.

**Krok 3:** Zaczynij od givenWeatherFor() i getWeather() - to jest fundament.

**Krok 4:** Skompiluj: ./gradlew compileDebugUnitTestKotlin.

**Krok 5:** Popraw błędy kompilacji.

**⚠ Uwaga:** Metody API testowego (givenWeatherFor, reset, fetchCount) NIE są częścią interfejsu WeatherRepository. Są to pomocnicze metody dostępne tylko z kodu testowego. Nie dodawaj ich do interfejsu produkcyjnego.

## Pisanie testów używających FakeWeatherRepository

```

// test/kotlin/.../HomeViewModelFakeTest.kt
// Ten test używa Fake zamiast Mock - sprawdź różnice w czytelności
class HomeViewModelFakeTest {

 private val fakeRepository = FakeWeatherRepository()
 private val getWeather = GetWeatherUseCase(fakeRepository)
 private val viewModel by lazy { HomeViewModel(getWeather) }
 private val testDispatcher = StandardTestDispatcher()
}

```

```
@BeforeEach
fun setUp() {
 Dispatchers.setMain(testDispatcher)
 fakeRepository.reset() // Czysty stan przed kazdym testem
}

@AfterEach
fun tearDown() { Dispatchers.resetMain() }

@Test
fun `searching twice uses cache, not network`() = runTest {
 // Przygotuj dane:
 fakeRepository.givenWeatherFor("Czestochowa",
 buildWeatherData(city = "Czestochowa", temp = 22.0))

 // Pierwsze wyszukiwanie:
 viewModel.searchCity("Czestochowa")
 advanceUntilIdle()

 // Drugie wyszukiwanie tego samego miasta:
 viewModel.searchCity("Czestochowa")
 advanceUntilIdle()

 // Fake liczy wywołania - sprawdź czy sieć była odpytana tylko raz:
 assertThat(fakeRepository.fetchCount).isEqualTo(1)
 // (drugie wywołanie powinno być serwowane z pamięci ViewModel)
}

@Test
fun `favorites flow reflects add and remove sequence`() = runTest {
 fakeRepository.givenWeatherFor("Gdynia", buildWeatherData("Gdynia", 18.0))
 fakeRepository.givenWeatherFor("Sopot", buildWeatherData("Sopot", 20.0))

 fakeRepository.getFavorites().test {
 assertThat(awaitItem()).isEmpty() // Stan początkowy: pusta lista

 fakeRepository.addFavorite("Gdynia")
 assertThat(awaitItem()).hasSize(1)

 fakeRepository.addFavorite("Sopot")
 assertThat(awaitItem()).hasSize(2)

 fakeRepository.removeFavorite("Gdynia")
 val remaining = awaitItem()
 assertThat(remaining).hasSize(1)
 assertThat(remaining.first().city).isEqualTo("Sopot")

 cancelAndIgnoreRemainingEvents()
 }
}
}
```

**Krok 1:** Napisz oba testy. Uruchom i sprawdź wyniki.

**Krok 2:** Sprawdź czytelność: porównaj z testem opartym na Mock. Który łatwiej zrozumieć?

✓ **Weryfikacja:** Testy z Fake są czytelne jak scenariusz: 'daj mi dane dla Gdyni, potem dodaj do ulubionych, sprawdź listę'. Brak coEvery i coVerify - logika jest w Fake.

## 17. Testowanie kompletnego stosu MVVM

Prawdziwa wartość testów ujawnia się gdy testujemy cały stos współpracujących klas: UseCase wywołujący Repository, ViewModel korzystający z UseCase, komponent UI reagujący na ViewModel. To ćwiczenie prowadzi przez testowanie każdej warstwy i ich integracji.

### Testowanie warstwy domenowej - Use Case

Use Case to centralny punkt logiki biznesowej. Test Use Case sprawdza czy logika transformacji, walidacji i decyzji jest poprawna - niezależnie od źródła danych i warstwy prezentacji.

```
// Kompletna klasa Use Case z własną logiką biznesową:
class GetWeatherUseCase(private val repository: WeatherRepository) {
 suspend operator fun invoke(city: String): Result<WeatherData> {
 if (city.isBlank()) return Result.failure(IllegalArgumentException("Pusta nazwa miasta"))
 if (city.length < 2) return Result.failure(IllegalArgumentException("Nazwa zbyt krótka"))
 val normalized = city.trim().replaceFirstChar { it.uppercase() }
 return repository.getWeather(normalized)
 }
}

// Testy walidacji wejścia - bez mocka, tylko logika:
class GetWeatherUseCaseValidationTest {

 // Używamy FakeRepository - nie Mock - bo chcemy też testować ścieżkę sukcesu
 private val fakeRepo = FakeWeatherRepository()
 private val useCase = GetWeatherUseCase(fakeRepo)

 @ParameterizedTest
 @ValueSource(strings = ["", " ", "\t"])
 fun `returns failure for blank city name`(input: String) = runTest {
 val result = useCase(input)
 assertThat(result.isFailure).isTrue()
 assertThat(result.exceptionOrNull())
 .isInstanceOf(IllegalArgumentException::class.java)
 }

 @Test
 fun `returns failure for single character city`() = runTest {
 val result = useCase("a")
 assertThat(result.isFailure).isTrue()
 }
}
```

```

@Test
fun `normalizes city name before passing to repository`() = runTest {
 fakeRepo.givenWeatherFor("Radom", buildWeatherData("Radom", 17.0))

 useCase(" radom ") // Ze spacjami i mala litera

 // Sprawdz ze po normalizacji city = 'Radom' (nie ' radom ')
 assertThat(fakeRepo.fetchCount).isEqualTo(1) // Wywolano raz
 // Gdyby normalizacja nie zadzialala, getWeather(' radom ') nie znalazloby da-
nych
}

@Test
fun `propagates repository error unchanged`() = runTest {
 val networkError = IOException("DNS resolution failed")
 fakeRepo.givenErrorFor("Nowy Sacz", networkError)

 val result = useCase("Nowy Sacz")

 assertThat(result.isFailure).isTrue()
 assertThat(result.exceptionOrNull()).isEqualTo(networkError)
}
}

```

**Krok 1:** Dodaj `@ParameterizedTest` do `build.gradle.kts`: `testImplementation('org.junit.jupiter:junit-jupiter-params:5.10.2')`.

**Krok 2:** Napisz testy walidacji. Uruchom z `./gradlew test --tests '*UseCaseValidation*'`.

**Krok 3:** Dodaj własny przypadek walidacji do Use Case (np. miasto nie może zawierać cyfr) i napisz test potwierdzający nowe zachowanie.

✓ **Weryfikacja:** Testy walidacji nie potrzebują mocków, bo sprawdzają czystą logikę Kotliną. Przypadki graniczne (blank, jedna litera) są pokryte przez `ParameterizedTest`.

## Testowanie ViewModel ze StateFlow

ViewModel łączy Use Case z UI przez StateFlow. Testowanie polega na weryfikacji sekwencji stanów emitowanych przez StateFlow w odpowiedzi na akcje użytkownika. Wymaga `TestDispatcher` i `Turbine`.

```

// Definicja stanu UI - sealed class:
sealed interface WeatherUiState {
 data object Idle : WeatherUiState
 data object Loading : WeatherUiState
 data class Success(val data: WeatherData) : WeatherUiState
 data class Error(val message: String) : WeatherUiState
}

// ViewModel emitujący stany przez StateFlow:
class HomeViewModel(private val getWeather: GetWeatherUseCase) : ViewModel() {

```

```
private val _uiState = MutableStateFlow<WeatherUiState>(WeatherUiState.Idle)
val uiState: StateFlow<WeatherUiState> = _uiState.asStateFlow()

fun searchCity(city: String) {
 viewModelScope.launch {
 _uiState.value = WeatherUiState.Loading
 getWeather(city)
 .onSuccess { _uiState.value = WeatherUiState.Success(it) }
 .onFailure { _uiState.value = WeatherUiState.Error(it.message ?: "Blad") }
 }
}

// Testy ViewModel z Turbine i FakeRepository:
class HomeViewModelTest {

 private val fakeRepo = FakeWeatherRepository()
 private val useCase = GetWeatherUseCase(fakeRepo)
 private val dispatcher = StandardTestDispatcher()

 @BeforeEach
 fun setUp() {
 Dispatchers.setMain(dispatcher)
 fakeRepo.reset()
 }

 @AfterEach
 fun tearDown() { Dispatchers.resetMain() }

 @Test
 fun `emits Loading then Success on successful search`() = runTest {
 fakeRepo.givenWeatherFor("Tarnow", buildWeatherData("Tarnow", 21.0))
 val viewModel = HomeViewModel(useCase)

 viewModel.uiState.test {
 // Stan poczatkowy - Idle:
 assertThat(awaitItem()).isInstanceOf(WeatherUiState.Idle::class.java)

 viewModel.searchCity("Tarnow")

 // Loading - korutyna startuje, ale nie zakonczyła jeszcze:
 assertThat(awaitItem()).isInstanceOf(WeatherUiState.Loading::class.java)

 // Uruchom korutyny do konca:
 advanceUntilIdle()

 // Success - dane sa dostepne:
 val success = awaitItem()
 assertThat(success).isInstanceOf(WeatherUiState.Success::class.java)
 assertThat((success as WeatherUiState.Success).data.city).isEqualTo("Tarnow")

 cancelAndIgnoreRemainingEvents()
 }
 }
}
```

```
@Test
fun `emits Loading then Error on network failure`() = runTest {
 fakeRepo.givenErrorFor("Przemysl", IOException("Timeout"))
 val viewModel = HomeViewModel(useCase)

 viewModel.uiState.test {
 awaitItem() // Idle
 viewModel.searchCity("Przemysl")
 awaitItem() // Loading
 advanceUntilIdle()
 val error = awaitItem()
 assertThat(error).assertInstanceOf(WeatherUiState.Error::class.java)
 assertThat((error as WeatherUiState.Error).message).contains("Timeout")
 cancelAndIgnoreRemainingEvents()
 }
}

@Test
fun `second search replaces first result`() = runTest {
 fakeRepo.givenWeatherFor("Jaworzno", buildWeatherData("Jaworzno", 20.0))
 fakeRepo.givenWeatherFor("Zabrze", buildWeatherData("Zabrze", 18.0))
 val viewModel = HomeViewModel(useCase)

 viewModel.uiState.test {
 awaitItem() // Idle

 viewModel.searchCity("Jaworzno")
 awaitItem() // Loading
 advanceUntilIdle()
 val first = awaitItem() as WeatherUiState.Success
 assertThat(first.data.city).isEqualTo("Jaworzno")

 viewModel.searchCity("Zabrze")
 awaitItem() // Loading ponownie
 advanceUntilIdle()
 val second = awaitItem() as WeatherUiState.Success
 assertThat(second.data.city).isEqualTo("Zabrze")

 cancelAndIgnoreRemainingEvents()
 }
}
}
```

**Krok 1:** Zaimplementuj sealed interface `WeatherUiState` i zaktualizuj `HomeViewModel`.

**Krok 2:** Napisz wszystkie trzy testy. Uruchom z `./gradlew test`.

**Krok 3:** Dodaj test dla przypadku: szukanie tego samego miasta dwa razy pod rząd. Drugi wynik powinien zastępować pierwszy.

**X Częsty błąd:** Bez `advanceUntilIdle()` po `searchCity()` coroutine nigdy się nie wykona, bo test zawiesza się czekając na `Success`, które nigdy nie nadejdzie. `StandardTestDispatcher` wymaga ręcznego popchnięcia czasu.

## Test integracyjny Use Case + Repository + DAO

Ten test łączy trzy prawdziwe klasy: `Use Case`, `RepositoryImpl` i `Room DAO` (in-memory). Nie ma żadnych mocków - tylko prawdziwy kod z bazą w pamięci.

```
// Inicjalizacja środowiska testowego:
@RunWith(AndroidJUnit4::class)
class GetWeatherUseCaseIntegrationTest {

 private lateinit var db: WeatherDatabase
 private lateinit var dao: WeatherDao
 private lateinit var repository: WeatherRepositoryImpl
 private lateinit var useCase: GetWeatherUseCase

 // FakeApi zastępuje tylko HTTP - reszta (Room, mapowanie) jest prawdziwa
 private val fakeApi = FakeWeatherApi()

 @BeforeEach
 fun setUp() {
 val ctx = ApplicationProvider.getApplicationContext<Context>()
 db = Room.inMemoryDatabaseBuilder(ctx, WeatherDatabase::class.java)
 .allowMainThreadQueries()
 .build()
 dao = db.weatherDao()
 repository = WeatherRepositoryImpl(fakeApi, dao)
 useCase = GetWeatherUseCase(repository)
 }

 @AfterEach
 fun tearDown() { db.close() }

 @Test
 fun `data from api is persisted in room after search`() = runTest {
 fakeApi.givenWeatherFor("Nowy Targ", FakeWeatherApi.successResponse(
 temp = 10.5, humidity = 90, code = 71
))

 // Wywołaj prawdziwy Use Case:
 val result = useCase("Nowy Targ")

 // Sprawdź wynik Use Case:
 assertThat(result.isSuccess).isTrue()

 // Sprawdź że Room naprawdę zapamiętał dane:
 val cached = dao.getWeatherForCity("Nowy Targ")
 assertThat(cached).isNotNull()
 assertThat(cached!!.temperature).isWithin(0.01).of(10.5)
 }
}
```



```

@Test
fun `second call serves data from cache without api call`() = runTest {
 fakeApi.givenWeatherFor("Krosno", FakeWeatherApi.successResponse(20.0, 55, 1))

 useCase("Krosno") // Pierwsze wywołanie - pobiera z API
 useCase("Krosno") // Drugie wywołanie - powinno wziac z cache

 // Sprawdz ze API zostalo odpytane tylko raz:
 assertThat(fakeApi.callCount).isEqualTo(1)
}
}

```

**Krok 1:** Stwórz FakeWeatherApi analogicznie do FakeWeatherRepository - implementuje interfejs WeatherApi, liczy wywołania.

**Krok 2:** Napisz oba testy integracyjne. Uruchom na emulatorze (./gradlew connectedAndroidTest).

✓ **Weryfikacja:** Test integracyjny potwierdza że trzy warstwy współpracują poprawnie: API → mapowanie DTO → zapis do Room → odczyt z cache.

## 18. Snapshot testing widoków Compose z Paparazzi

Testy snapshot (inaczej: golden tests lub visual regression tests) wykonują zrzut ekranu komponentu UI i porównują go ze wzorcem zapisanym w repozytorium. Jeśli wygląd komponentu zmieni się nieoczekiwanie, test zakłada. Paparazzi (Cash App) renderuje komponenty Compose bez emulatora - na JVM.

### Analogia: fotografia i negatyw.

Pierwsze uruchomienie testu Paparazzi robi zdjęcie komponentu (golden image). Każde następne uruchomienie porównuje nowe zdjęcie z negatywem: identyczne - test zielony, różnica pikselowa - test czerwony. Gdy zmiana wyglądu jest celowa (np. nowy kolor branding), akceptujesz nowy negatyw: ./gradlew recordPaparazziDebug.

## Instalacja i pierwsza fotografia

```

build.gradle.kts (poziom projektu):
plugins {
 id("app.cash.paparazzi") version "1.3.4" apply false
}

build.gradle.kts (poziom modulu :app):
plugins {
 id("app.cash.paparazzi")
}

Sync Gradle. Paparazzi dodaje automatycznie zadania:
./gradlew testDebugUnitTest - uruchamia testy snapshot

```

```
./gradlew recordPaparazziDebug - aktualizuje golden images
./gradlew verifyPaparazziDebug - tylko weryfikacja (używane w CI)
```

**Krok 1:** Dodaj plugin Paparazzi do obu plików build.gradle.kts. Zsynchronizuj Gradle.

**Krok 2:** Sprawdź, że nowe zadania Gradle są dostępne: `./gradlew tasks | grep -i paparazzi`

✓ **Weryfikacja:** Trzy zadania Paparazzi widoczne w liście zadań Gradle.

## Pisanie testów snapshot dla WeatherCard

```
// test/kotlin/.../snapshot/WeatherCardSnapshotTest.kt
// Uwaga: katalog test/ (nie androidTest/) - Paparazzi działa na JVM!

@RunWith(RobolectricTestRunner::class)
class WeatherCardSnapshotTest {

 @get:Rule
 val paparazzi = Paparazzi(
 deviceConfig = DeviceConfig.PIXEL_6, // Symulowane urządzenie
 theme = "android:Theme.Material.Light.NoActionBar",
 showSystemUi = false, // Tylko komponent, bez paska statusu
)

 private val clearWeather = WeatherData(
 city = "Warszawa", temperature = 22.5,
 humidity = 60, windSpeed = 10.0, weatherCode = 0
)

 private val stormyWeather = WeatherData(
 city = "Gdansk", temperature = 14.0,
 humidity = 95, windSpeed = 45.0, weatherCode = 99
)

 @Test
 fun weatherCard_clearSky() {
 paparazzi.snapshot {
 AppTheme {
 WeatherCard(
 weather = clearWeather,
 onPress = {},
)
 }
 }
 }

 @Test
 fun weatherCard_storm() {
 paparazzi.snapshot {
 AppTheme {
 WeatherCard(weather = stormyWeather, onPress = {})
 }
 }
 }
}
```

```
@Test
fun weatherCard_darkTheme() {
 paparazzi.snapshot {
 // Compose dark mode przez CompositionLocalProvider:
 AppTheme(darkTheme = true) {
 WeatherCard(weather = clearWeather, onPress = {})
 }
 }
}

@Test
fun weatherCard_longCityName() {
 val longName = clearWeather.copy(
 city = "Wieliszew nad Narwia z dluga nazwa"
)
 paparazzi.snapshot {
 AppTheme { WeatherCard(weather = longName, onPress = {}) }
 }
}
```

**Krok 1:** Stwórz klasę WeatherCardSnapshotTest.

**Krok 2:** Uruchom po raz pierwszy: `./gradlew recordPaparazziDebug`

**Krok 3:** Sprawdź wygenerowane golden images w katalogu: `src/test/snapshots/images/`

**Krok 4:** Wprowadź małe zmiany do WeatherCard.kt (np. zmien kolor tła).

**Krok 5:** Uruchom: `./gradlew verifyPaparazziDebug`

**Krok 6:** Obserwuj komunikat błędu i porównanie pikselowe.

**Krok 7:** Cofnij zmianę lub zaakceptuj nowy wygląd: `./gradlew recordPaparazziDebug`

✓ **Weryfikacja:** Golden images są w katalogu snapshots/. Test verifyPaparazzi wykrywa zmianę pixelową i failuje. Cofnięcie zmiany sprawia że test znowu jest zielony.

✗ **Częsty błąd:** Paparazzi generuje pliki PNG które MUSZA być commitowane do repozytorium (git add src/test/snapshots/). Bez nich CI zawsze failuje z brakiem wzorca do porównania.

## Integracja Paparazzi z GitHub Actions

```
W ci.yml - dodaj job weryfikujący snapshots:
snapshot-tests:
 name: Snapshot Tests (Paparazzi)
 runs-on: ubuntu-latest
 needs: static-analysis
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-java@v4
 with: { java-version: '17', distribution: 'temurin', cache: 'gradle' }
 - run: chmod +x gradlew
```

```
- name: Weryfikacja snapshot (brak aktualizacji golden images w CI)
 run: ./gradlew verifyPaparazziDebug
- name: Artefakt z roznicami (jesli test nie przejdzie)
 uses: actions/upload-artifact@v4
 if: failure()
 with:
 name: paparazzi-failures
 path: app/build/paparazzi/failures/
 retention-days: 5
```

**Krok 1:** Dodaj job snapshot-tests do .github/workflows/ci.yml.

**Krok 2:** Upewnij się że pliki golden images są committed (git add src/test/snapshots/).

**Krok 3:** Wypchnij zmiany i obserwuj job na GitHub Actions.

## 19. Testy wydajności: Android Macrobenchmark

Macrobenchmark mierzy czas uruchomienia aplikacji (startup time), czas renderowania list (scrolling) i reakcje na akcje użytkownika. Wyniki są precyzyjne i powtarzalne, bo framework izoluje wpływ systemu operacyjnego, GC i cieplnej regulacji procesora. Testy uruchamiają się na fizycznym urządzeniu lub emulatorze.

### Konfiguracja modułu benchmark

```
Macrobenchmark wymaga osobnego modułu Android Library:
File > New > New Module > Benchmark Module (macrobenchmark)
Nazwa modułu: :macrobenchmark

macrobenchmark/build.gradle.kts:
plugins {
 id("com.android.test")
 id("org.jetbrains.kotlin.android")
}

android {
 targetProjectPath = ":app" // Moduł do benchmarkowania
 experimentalProperties["android.experimental.self-instrumenting"] = true

 defaultConfig {
 testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
 testInstrumentationRunnerArguments["androidx.benchmark.suppressErrors"] =
 "EMULATOR,DEBUGGABLE" // Pozwól na uruchomienie na emulatorze (mniej dokładne)
 }

 buildTypes {
 create("benchmark") {
 isDebuggable = false
 signingConfig = signingConfigs.getByName("debug")
 matchingFallbacks += listOf("release")
 }
 }
}
```

```
dependencies {
 implementation("androidx.test.ext:junit:1.1.5")
 implementation("androidx.benchmark:benchmark-macro-junit4:1.2.4")
 implementation("androidx.test.uiautomator:uiautomator:2.3.0")
}
```

**Krok 1:** Utwórz modul macrobenchmark przez File > New > New Module > Benchmark.

**Krok 2:** Dodaj android:profileable = true do AndroidManifest.xml aplikacji (w sekcji <application>).

✓ **Weryfikacja:** Modul :macrobenchmark skonfigurowany. Gradle sync przechodzi bez błędów.

## Pomiar czasu uruchomienia aplikacji

```
// macrobenchmark/src/androidTest/kotlin/.../StartupBenchmark.kt
@RunWith(AndroidJUnit4::class)
class StartupBenchmark {

 @get:Rule
 val benchmarkRule = MacrobenchmarkRule()

 // Pomiar czasu 'zimnego startu' (cold start) - aplikacja nie była uruchomiona
 @Test
 fun coldStartup() = benchmarkRule.measureRepeated(
 packageName = "pl.edu.pam.weather",
 metrics = listOf(StartupTimingMetric()),
 iterations = 5, // Powroz 5 razy, bierz mediane
 startupMode = StartupMode.COLD, // Zimny start: zabij proces przed kazdym te-
stem
 setupBlock = {
 // Uruchom i zabij aplikacje - upewnij sie ze cache jest czysty
 pressHome()
 killProcess()
 },
) {
 // Mierzone dzialanie: uruchom aplikacje i poczekaj az HomeScreen jest widoczny:
 startActivityAndWait() // Uruchamia aktywnosc i czeka az renderowanie sie zakon-
czy
 }

 // Pomiar 'cieplego startu' (warm start) - aplikacja w tle
 @Test
 fun warmStartup() = benchmarkRule.measureRepeated(
 packageName = "pl.edu.pam.weather",
 metrics = listOf(StartupTimingMetric()),
 iterations = 10,
 startupMode = StartupMode.WARM,
 setupBlock = { pressHome() },
) {
 startActivityAndWait()
 }
}
```

```
// Pomiar płynności przewijania listy:
class ScrollBenchmark {

 @get:Rule val benchmarkRule = MacrobenchmarkRule()

 @Test
 fun scrollFavoritesList() = benchmarkRule.measureRepeated(
 packageName = "pl.edu.pam.weather",
 metrics = listOf(FrameTimingMetric()), // Mierzy czas renderowania klatek
 iterations = 5,
 startupMode = StartupMode.WARM,
 setupBlock = {
 val intent = Intent("pl.edu.pam.weather.MAIN").apply {
 setPackage("pl.edu.pam.weather")
 }
 startActivityAndWait(intent)
 },
) {
 val list = device.findObject(By.res("pl.edu.pam.weather:id/favorites_list"))
 // Przewijaj listę 5 razy:
 repeat(5) { list.fling(Direction.DOWN) }
 }
}
```

**Krok 1:** Napisz StartupBenchmark. Uruchom na emulatorze: ./gradlew

:macrobenchmark:connectedBenchmarkAndroidTest

**Krok 2:** Sprawdź wyniki w Android Studio > Run > Benchmark Results.

**Krok 3:** Zanotuj medianę czasu zimnego startu.

**Krok 4:** Porównaj wyniki na tle rekomendacji Google: cold start < 500 ms, warm start < 200 ms.

**⚠ Uwaga:** Wyniki na emulatorze są 3-5x gorsze, niż na fizycznym urządzeniu. Benchmarking na emulatorze służy do wykrywania regresji między wersjami, nie do bezwzględnych pomiarów.

**Best practice:** Wbuduj benchmark w CI: uruchom benchmarki na pull requestie i porównaj z wartościami z gałęzi main. Jeśli czas uruchomienia wzrósł o > 20%, zgłoś komentarz w PR jako błąd wydajności.

## 20. Zaawansowany GitHub Actions: macierze i artefakty

Podstawowe workflow CI uruchamia jeden job na jednym systemie. Zaawansowane techniki pozwalają testować równoległe na wielu wersjach API Androida, automatycznie publikować artefakty, cache'ować zależności i skrócić czas budowania o 60-80%.

## Macierze buildów: testowanie na wielu wersjach Android

Macierze (matrix) uruchamiają ten sam job z różnym zestawem parametrów równoległe. Dzięki temu w jednym workflow można przetestować aplikację na API 29, 31, 33 i 34 jednocześnie, zamiast sekwencyjnie.

```
.github/workflows/ci-matrix.yml
name: CI z matryca API

on: [push, pull_request]

jobs:
 instrumented-tests:
 name: Testy API ${matrix.api-level} (${matrix.arch})
 runs-on: ubuntu-latest

 strategy:
 fail-fast: false # Nie przerywaj wszystkich jobów gdy jeden failuje
 matrix:
 api-level: [29, 31, 33, 34]
 arch: [x86_64]
 include:
 # Dodaj dodatkowa kombinacje dla starszej architektury:
 - api-level: 29
 arch: x86
 exclude:
 # Wyklucz niezgodna kombinacje:
 - api-level: 34
 arch: x86

 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-java@v4
 with: { java-version: '17', distribution: 'temurin', cache: 'gradle' }
 - run: chmod +x gradlew

 - name: Cache AVD
 uses: actions/cache@v4
 id: avd-cache
 with:
 path: |
 ~/.android/avd/*
 ~/.android/adb*
 key: avd-${matrix.api-level}-${matrix.arch}

 - name: Stworz AVD (jesli brak w cache)
 if: steps.avd-cache.outputs.cache-hit != 'true'
 uses: reactivecircus/android-emulator-runner@v2
 with:
 api-level: ${matrix.api-level}
 arch: ${matrix.arch}
 force-avd-creation: false
 emulator-options: -no-window -gpu swiftshader_indirect -noaudio -no-boot-anim
```

```

 disable-animations: false
 script: echo 'AVD gotowe'

- name: Uruchom testy
 uses: reactivecircus/android-emulator-runner@v2
 with:
 api-level: ${{ matrix.api-level }}
 arch: ${{ matrix.arch }}
 force-avd-creation: false
 emulator-options: -no-window -gpu swiftshader_indirect -noaudio -no-boot-anim
 disable-animations: true
 script: ./gradlew connectedDebugAndroidTest

- name: Wysluj wyniki
 uses: actions/upload-artifact@v4
 if: always()
 with:
 name: test-results-api${{ matrix.api-level }}-${{ matrix.arch }}
 path: app/build/reports/androidTests/
 retention-days: 7

```

**Krok 1:** Dodaj plik ci-matrix.yml do .github/workflows/.

**Krok 2:** Wypchnij commit i obserwuj na GitHub Actions: czy joby uruchamiają się równolegle?

**Krok 3:** Sprawdź widok matrycy: GitHub pokazuje tabele API level x arch z indywidualnymi statusami.

✓ **Weryfikacja:** Cztery joby (API 29, 31, 33, 34) uruchamiają się równolegle. fail-fast: false oznacza że job API 33 kontynuuje nawet jeśli API 29 failuje.

## Zaawansowane zarządzanie cache Gradle

Cache zależności Gradle to jeden z najważniejszych elementów optymalizacji CI. Bez cache, pierwsze uruchomienie workflow pobiera kilkaset MB bibliotek. Z cache: tylko co się zmieniło.

```

Optymalna konfiguracja cache w workflow:
- name: Cache Gradle caches i wrapper
 uses: actions/cache@v4
 with:
 path: |
 ~/.gradle/caches
 ~/.gradle/wrapper
 ~/.android/build-cache
 # Klucz bazuje na OS + hash plików konfiguracyjnych Gradle:
 key: ${{ runner.os }}-gradle-${{ hashFiles('**/*.gradle*', '**/gradle-wrapper.properties', '**/libs.versions.toml') }}
 # Jeśli klucz nie pasuje, spróbuj starszy (bez hash):
 restore-keys: |
 ${{ runner.os }}-gradle-

Gradle Build Cache (oddzielny od cache zależności):
gradle.properties w projekcie:
org.gradle.caching=true

```



```
org.gradle.configuration-cache=true
org.gradle.parallel=true
org.gradle.daemon=false # Wyłącz daemon w CI (każdy job to świeży kontener)

Pomiar efektu cache: porównaj czasy w GitHub Actions:
- Pierwsze uruchomienie (brak cache): ~8-12 minut
- Drugie uruchomienie (pełny cache): ~2-4 minuty
- Zmiana kodu bez nowych zależności: ~1-2 minuty (tylko rebuild moduły)
```

**Krok 1:** Dodaj blok cache do swojego ci.yml.

**Krok 2:** Dodaj org.gradle.caching=true do gradle.properties.

**Krok 3:** Wypchnij dwa identyczne commity jeden po drugim.

**Krok 4:** Porównaj czas budowania pierwszego i drugiego.

✓ **Weryfikacja:** Drugi build jest 2-4x szybszy. W logu widac 'Cache hit' zamiast 'Cache miss'.

## Kondycyjne uruchamianie jobow

```
Zaawansowane warunki uruchamiania jobow:
jobs:
 # Uruchom tylko jesli zmienily sie pliki Kotlin (nie README, nie docs):
 unit-tests:
 if: |
 github.event_name == 'push' ||
 contains(github.event.pull_request.labels.*.name, 'run-tests')
 # Alternatywnie: filtruj sciezki

 # Lub uzywaj paths w triggerze on: push:
 # on:
 # push:
 # paths:
 # - '**.kt'
 # - '**.gradle*'
 # - '!**.md' # Wyklucz pliki markdown

 # Job uruchamiany TYLKO na main (nie na PR):
 deploy:
 if: github.ref == 'refs/heads/main' && github.event_name == 'push'
 needs: [unit-tests, static-analysis]
 runs-on: ubuntu-latest
 steps:
 - run: echo 'Budowanie release APK...'

 # Job pomijajacy gdy PR jest draft:
 expensive-test:
 if: github.event.pull_request.draft == false
 runs-on: ubuntu-latest
 steps:
 - run: echo 'Długi test (tylko dla gotowych PR)'
```

**Krok 1:** Dodaj warunek if: do jobu instrumented-tests tak, aby uruchamiał się tylko na main lub gdy PR ma etykietę 'run-e2e'.

**Krok 2:** Otwórz PR jako draft i sprawdź, że expensive-test jest pomijany. Zmień na 'ready for review' i obserwuj.

## Automatyczne wydania z semantic-release

Semantic-release to narzędzie, które automatycznie: analizuje commity od ostatniego wydania (na podstawie Conventional Commits), oblicza numer następnej wersji (semver), generuje CHANGELOG, tworzy Git tag, tworzy GitHub Release z listem zmian i opcjonalnie buduje i dołącza artefakty (APK).

## Konfiguracja semantic-release

```
Instalacja:
npm install --save-dev semantic-release
npm install --save-dev @semantic-release/changelog
npm install --save-dev @semantic-release/git
npm install --save-dev @semantic-release/github

.releaserc.json - konfiguracja semantic-release:
{
 "branches": ["main"],
 "plugins": [
 ["@semantic-release/commit-analyzer", {
 "preset": "conventionalcommits",
 "releaseRules": [
 {"type": "feat", "release": "minor"},
 {"type": "fix", "release": "patch"},
 {"type": "perf", "release": "patch"},
 {"type": "refactor", "release": false}, // Refactor nie powoduje wydania
 {"type": "docs", "release": false},
 {"type": "chore", "release": false},
 {"breaking": true, "release": "major"}
]
 }],
 ["@semantic-release/release-notes-generator", {
 "preset": "conventionalcommits"
 }],
 ["@semantic-release/changelog", {
 "changelogFile": "CHANGELOG.md"
 }],
 ["@semantic-release/git", {
 "assets": ["CHANGELOG.md"],
 "message": "chore(release): ${nextRelease.version} [skip ci]"
 // [skip ci] zapobiega wywołaniu CI przez automatyczny commit
 }],
 ["@semantic-release/github", {
 "assets": [
 {"path": "app/build/outputs/apk/release/app-release.apk",
 "label": "WeatherApp-${nextRelease.version}.apk"}
]
 }],
]
}
```

```
}
```

**Krok 1:** Utwórz plik `.releaserc.json` w katalogu głównym projektu.

**Krok 2:** Zainstaluj `semantic-release`: `npm install --save-dev semantic-release` (wymaga `package.json` - utwórz pustym `npm init -y` jeśli nie ma).

✓ **Weryfikacja:** Plik `.releaserc.json` i `node_modules/` są dostępne. Dodaj `node_modules/` do `.gitignore`.

## Workflow GitHub Actions dla automatycznych wydań

```
.github/workflows/release.yml
name: Automatyczne wydanie

on:
 push:
 branches: [main]

permissions:
 contents: write # Tworzenie tagów i release
 issues: write # Komentarze do issues
 pull-requests: write # Komentarze do PR

jobs:
 release:
 name: Semantic Release
 runs-on: ubuntu-latest
 # Uruchom tylko jeśli to nie jest commit od semantic-release:
 if: "!contains(github.event.head_commit.message, '[skip ci]')"

 steps:
 - uses: actions/checkout@v4
 with:
 fetch-depth: 0 # WYMAGANE: pełna historia dla semantic-release
 persist-credentials: false

 - uses: actions/setup-java@v4
 with: { java-version: '17', distribution: 'temurin', cache: 'gradle' }

 - uses: actions/setup-node@v4
 with: { node-version: '20' }

 - run: npm ci # Instalacja semantic-release i pluginów
 - run: chmod +x gradlew

 - name: Budowanie release APK
 run: ./gradlew assembleRelease

 - name: Uruchomienie semantic-release
 env:
```

```
GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }}
GITHUB_TOKEN jest automatycznie dostępny - nie musisz go dodawać!
run: npx semantic-release
```

**Krok 1:** Dodaj plik release.yml. Wypchnij commit z typem feat: (np. feat(ui): dodaj animacje przejsc).

**Krok 2:** Obserwuj workflow na GitHub Actions. Sprawdź czy: CHANGELOG.md został zaktualizowany, tag git został stworzony (np. v1.1.0), GitHub Release pojawił się z listem zmian.

**Krok 3:** Wypchnij następnie commit z typem fix: i obserwuj czy wersja wzrosła z 1.1.0 do 1.1.1.

✓ **Weryfikacja:** Po feat: wersja wzrasta do v1.x.0. Po fix: do v1.x.1. GitHub Release zawiera CHANGELOG z listą zmian.

⚠ **Uwaga:** semantic-release wymaga fetch-depth: 0 w checkout. Bez pełnej historii nie może znaleźć poprzedniego tagu i obliczyć wersji.

## 21. Code review w praktyce: narzędzia i workflow

Code review to jedna z najważniejszych praktyk inżynierskich. Nie chodzi o sprawdzanie błędów kompilacji (robi to CI), ale o wymianę wiedzy, wykrywanie problemów projektowych i utrzymanie spójności kodu. To ćwiczenie symuluje pełny cykl review w parach.

### Przygotowanie PR do review: checklista autora

Dobry PR jest łatwy do przejrzania. Nie przekracza 400 linii zmian, ma jeden dobrze opisany cel i zawiera dowody działania (testy, screenshot). Zły PR to 2000-liniowe monolity z kilkoma niezwiązanymi zmianami.

Element	Jak sprawdzić	Dlaczego ważne
Rozmiar PR < 400 linii	GitHub pokazuje '+X -Y lines changed' w nagłówku PR.	Większy PR = trudniejszy review = więcej pominiętych błędów. Podziel duże PR na mniejsze.
Jeden cel, jeden PR	Tytuł PR powinien opisywać jedną zmianę. Nie: 'fix bug + add feature + refactor'.	Łatwe cofnięcie (revert) gdy zmiana okaże się błędna.
Testy do każdej nowej logiki	CI musi być zielone. Coverage nie może spaść.	Reviewer nie musi ręcznie weryfikować zachowania; testy to robią.
Self-review przed otwarciem PR	Przejrzyj własne diff na GitHub zanim poprosisz o review.	Eliminuje oczywiste błędy (zapomniane debug logi, zakomentowany kod).
Opis PR: co i dlaczego	Uzupełnij szablon PR: opis zmian, powiązany Issue, kroki testowania.	Reviewer bez kontekstu pyta o rzeczy, które autor zna. Opis eliminuje pytania.

### Symulacja code review w parach

Ćwiczenie wykonujemy w parach: osoba A otwiera PR z kodem do sprawdzenia (przygotowanym przez prowadzącego), osoba B przeprowadza review. Potem zamieniacie się rolami.

**Instrukcja:** Kod do review zostanie udostępniony przez prowadzącego jako branch do sklonowania. Znajdziesz w nim celowe problemy: zła organizacja kodu, brakujące testy, potencjalne NPE, niezgodne z konwencją nazewnictwa.

```
Osoba A: sklonuj branch z kodem do review:
git fetch origin review/exercise-code
git checkout review/exercise-code
Otwórz PR z tego brancha do main swojego repozytorium

Osoba B: przeprowadz review na GitHub
Kliknij 'Files changed' w PR i komentuj poszczególne linie.

Typy komentarzy w GitHub Review:
[Suggestion] - proponuj konkretną poprawkę przez blok 'Suggested Change':
```suggestion
val normalized = city.trim().lowercase()
```

[Question] - pytaj o cel kodu:
'Czy ta metoda musi być public? Wydaje się że używa jej tylko WeatherViewModel.'

[Nit] - małe problemy nie blokujące scalenia:
'nit: zmienna powinna mieć nazwę 'weatherData' zamiast 'data' (za ogólna)'

[Blocker] - problem który MUSI być naprawiony przed scaleniem:
'Blocker: brak obsługi błędów HTTP. Jeśli API zwróci 500, aplikacja crasha.'

Zakonczenie review: 'Submit review' > 'Request changes' lub 'Approve'
```

**Krok 1:** Osoba A: otwórz PR z kodem do review (branch udostępniony przez prowadzącego).

**Krok 2:** Osoba B: przejrzyj PR na GitHub. Znajdź min. 3 problemy i zostaw komentarze z kategoriami ([Blocker], [Nit], [Suggestion]).

**Krok 3:** Osoba A: odpowiedz na komentarze i popraw kod w odpowiednim commitcie. Oznacz komentarze jako 'Resolved'.

**Krok 4:** Osoba B: zaakceptuj poprawki lub zostaw kolejne komentarze. Gdy wszystko OK: Approve.

**Krok 5:** Zamieńcie się rolami.

✓ **Weryfikacja:** Oba PR przeszły review z min. 3 komentarzami i poprawkami przed scaleniem.

Dobry komentarz review	Zły komentarz review
'Ta metoda ma zbyt wiele odpowiedzialności (SRP). Rozdziel logikę HTTP od logiki parsowania.'	'Ten kod jest brzydki.'
'Sugestia: używaj immutable list - List<> zamiast MutableList<> w zwracanym typie.'	'Nie lubię jak piszesz testy.'
'Blocker: brak null check dla response.body() - może rzucić NPE na liniach 45-47.'	'Dlaczego tak to napisałeś???'

'Pytanie: czy zamierzamy cache'owac odpowiedz 404? Wydaje mi sie ze to nie ma sensu.'

'Napisz komentarze do kodu.'

## 22. Bezpieczeństwo repozytorium: skanowanie zależności

Aplikacja mobilna to nie tylko kod, który piszesz, ale także dziesiątki bibliotek z npm/Gradle, które mogą zawierać podatności bezpieczeństwa (CVE). GitHub oferuje trzy darmowe narzędzia do automatycznego wykrywania i naprawiania podatności: Dependabot, CodeQL i Secret Scanning.

### Aktywacja GitHub Dependabot

```
.github/dependabot.yml - konfiguracja automatycznych aktualizacji:
version: 2
updates:
 # Zaleznosci Gradle (Android):
 - package-ecosystem: 'gradle'
 directory: '/' # Katalog z build.gradle.kts
 schedule:
 interval: 'weekly' # Sprawdzaj co tydzien
 day: 'monday' # W poniedzialek
 time: '08:00'
 open-pull-requests-limit: 5 # Max 5 otwartych PR naraz
 commit-message:
 prefix: 'chore(deps)' # Prefix dla Conventional Commits
 reviewers:
 - 'prowadzacy-login' # Automatycznie przypisz do review
 labels:
 - 'dependencies'
 - 'automated'
 ignore:
 # Ignoruj aktualizacje major (1.x -> 2.x) - robimy recznie:
 - dependency-name: 'androidx.*'
 update-types: ['version-update:semver-major']

 # Zaleznosci GitHub Actions:
 - package-ecosystem: 'github-actions'
 directory: '/'
 schedule:
 interval: 'monthly'
 commit-message:
 prefix: 'ci(actions)'
```

**Krok 1:** Utwórz plik `.github/dependabot.yml` i zcommituj do main.

**Krok 2:** Przejdź na GitHub > Security > Dependabot alerts. Sprawdź, czy są jakieś istniejące podatności w projekcie.

**Krok 3:** Sprawdź PR otwarte przez Dependabota (jeżeli są) w zakładce Pull Requests.

✓ **Weryfikacja:** Dependabot jest skonfigurowany. W zakładce Security > Dependabot alerts widac liste podatnosci (lub 'No alerts').

## CodeQL: analiza kodu pod kątem podatności

CodeQL to semantyczny analizator kodu Google/GitHub, który wykrywa podatności takie jak SQL injection, XSS, Path traversal, Insecure cryptography. Dla Kotliny obsługuje klasy podatności OWASP Mobile Top 10.

```
.github/workflows/codeql.yml
name: CodeQL Security Analysis

on:
 push:
 branches: [main]
 pull_request:
 branches: [main]
 schedule:
 - cron: '0 3 * * 1' # Co poniedziałek o 3:00 (poza godzinami szczytu CI)

jobs:
 analyze:
 name: Analiza CodeQL
 runs-on: ubuntu-latest
 permissions:
 security-events: write # Zapisywanie wyników SARIF do GitHub
 actions: read
 contents: read

 steps:
 - uses: actions/checkout@v4

 - name: Inicjalizacja CodeQL
 uses: github/codeql-action/init@v3
 with:
 languages: java-kotlin # Język analizy
 queries: security-extended # Rozszerzony zestaw reguł (wiecej detekcji)

 - uses: actions/setup-java@v4
 with: { java-version: '17', distribution: 'temurin', cache: 'gradle' }

 - run: chmod +x gradlew && ./gradlew assembleDebug

 - name: Uruchomienie analizy CodeQL
 uses: github/codeql-action/analyze@v3
 with:
 category: '/language:java-kotlin'
```

**Krok 1:** Utwórz plik codeql.yml i zcommituj.

**Krok 2:** Przejdź na GitHub > Security > Code scanning alerts. Poczekaj, aż analiza się zakończy.

**Krok 3:** Jeżeli są znalezione problemy (alerts), kliknij w jeden i przeczytaj opis podatności i sugerowane rozwiązanie.

✓ **Weryfikacja:** CodeQL workflow jest zielony. Zakładka Code scanning alerts zawiera wyniki analizy.

## Secret Scanning i zapobieganie wyciekom kluczy

```
GitHub automatycznie skanuje każdy commit w poszukiwaniu:
- Kluczy API (Google, AWS, Firebase, Stripe, GitHub tokens)
- Certyfikatów prywatnych (RSA, EC)
- Hasel baz danych w postaci plaintext

Aktywacja (bezpłatna dla publicznych repozytoriów):
Settings > Security > Secret scanning > Enable

Jeśli klucz zostanie znaleziony: GitHub wysyła e-mail i blokuje push (Push Protec-
tion).

Zapobieganie wyciekowi kluczy - dobre praktyki:
1. Zawsze używa zmiennych środowiskowych:
val apiKey = System.getenv("WEATHER_API_KEY") ?: BuildConfig.API_KEY

2. Klucze w local.properties (poza repozytorium):
local.properties:
MAPS_API_KEY=AiZaSyXXXXXXXXXXXXXXXXXX

build.gradle.kts - wczytaj z local.properties:
val localProperties = Properties().apply {
 load(rootProject.file("local.properties").InputStream())
}
buildConfigField("String", "MAPS_API_KEY",
 "'${localProperties["MAPS_API_KEY"]}'")

3. Dla CI: klucze jako GitHub Secrets, nigdy w kodzie.

Test: czy secret scanning jest aktywny - celowo dodaj 'fake' klucz:
github_pat_11ABCDEF (nieprawdziwy token GitHub)
GitHub powinien wykryć to w ciągu minut i wysłać alert.
```

**Krok 1:** Przejdź na Settings > Security > Secret scanning i upewnij się, że jest włączony.

**Krok 2:** Sprawdź local.properties, czy nie zawiera kluczy API. Sprawdź .gitignore, czy local.properties jest wymienione.

## 23. Kompletny sprint: od Issue do scalenia z main

Ostatnie ćwiczenie integruje wszystko z obu części instrukcji w realny cykl pracy: od zgłoszenia problemu przez Issue, przez implementację z testami na nowym branchu, code review, zielone CI, aż po scalenie do main z automatycznym tagiem wydania.



**Cel ćwiczenia**

Przejdź pełny cykl feature development w ciągu 45 minut, doświadczając każdego kroku profesjonalnego workflow. Po zakończeniu repozytorium będzie mieć: zamknięty Issue, scalony PR z review, zielone CI i nowy tag wersji.

**Krok 1. Tworzenie Issue**

```
Na GitHub otwórz nowe Issue używając szablonu bug_report.md:
Tytuł: feat: dodaj wyświetlanie prędkości wiatru w WeatherCard

Wypełnij szablon:
Opis zmiany
Karta pogody (WeatherCard) nie wyświetla prędkości wiatru.
Użytkownik nie ma pełnych informacji o pogodzie.

Oczekiwane zachowanie
WeatherCard powinna pokazywać prędkość wiatru w km/h pod temperatura.

Kryteria akceptacji
- [] WeatherCard wyświetla windSpeed z WeatherData
- [] Jednostka 'km/h' jest widoczna
- [] Dodany test snapshot Paparazzi pokazujący nowy wygląd
- [] Test jednostkowy weryfikujący że wartość jest wyświetlana

Przypisz Issue do siebie (Assignees) i dodaj etykietę 'feature'.
```

**Krok 1:** Otwórz nowe Issue na GitHub. Wypełnij wszystkie pola szablonu.

**Krok 2:** Zanotuj numer Issue (np. #12) - będzie używany w nazwie brancha i tytule PR.

✓ **Weryfikacja:** Issue jest otwarte, przypisane i z etykieta.

**Krok 2. Implementacja na nowym branchu**

```
Stwórz branch związany z Issue:
git checkout main
git pull origin main
git checkout -b feature/12-wind-speed-in-weather-card
^^
Konwencja: feature/NUMER_ISSUE-krotki-opis (kebab-case)

1. Zaktualizuj WeatherCard.kt - dodaj wyświetlanie predkosci wiatru
2. Napisz test jednostkowy:
- Sprawdź ze WeatherCard wyświetla tekst '12.0 km/h'
- Sprawdź ze jednostka 'km/h' jest widoczna
3. Uruchom snapshot Paparazzi i zaktualizuj golden image:
./gradlew recordPaparazziDebug
4. Upewnij się ze wszystkie testy przechodzą:
./gradlew test
5. Commity z Conventional Commits:
git add -A
git commit -m "feat(ui): dodaj predkosc wiatru do WeatherCard"
```

```
git commit -m "test(ui): dodaj testy WeatherCard z wiatrem"
git commit -m "test(snapshot): aktualizuj golden image dla WeatherCard"
```

**Krok 1:** Stwórz branch i zaimplementuj zmianę (dodanie windSpeed do WeatherCard).

**Krok 2:** Napisz test jednostkowy potwierdzający wyświetlanie prędkości wiatru.

**Krok 3:** Zaktualizuj golden image Paparazzi.

**Krok 4:** Upewnij się że ./gradlew test jest zielony. Wypchnij branch: git push -u origin HEAD

✓ **Weryfikacja:** Branch jest na GitHubie. Wszystkie testy lokalnie przechodzą.

### Krok 3. Otworzenie i review PR

```
Otwórz PR na GitHub:
1. GitHub automatycznie zaproponuje PR z twojego brancha
2. Tytuł: feat(ui): dodaj prędkość wiatru do WeatherCard (#12)
Wzorzec: <typ>(<zakres>): <opis> (#numer_issue)
3. Wypełnij szablon PR:

Opis zmian
Dodano wyświetlanie prędkości wiatru w km/h w komponencie WeatherCard.

Powiązany Issue
Closes #12 # To magiczne słowo automatycznie zamknie Issue po scaleniu!

Checklist
- [x] Kod kompiluje się bez ostrzeżeń
- [x] Dodano test jednostkowy dla nowej logiki
- [x] Zaktualizowano snapshot Paparazzi
- [x] Wszystkie testy przechodzą

Poproś partnera z ćwiczenia 8 o review:
Reviewers > szukaj po loginie > Add

Obserwuj GitHub Actions - czy CI przeszło?
Jeśli nie: popraw błędy, push nowy commit, CI uruchomi się ponownie.
```

**Krok 1:** Otwórz PR na GitHub. Wypełnij szablon. Dodaj reviewera.

**Krok 2:** Poczekaj na review partnera. Jeśli reviewers nie ma, zrób self-review i zaznacz 'Approve'.

**Krok 3:** Sprawdź czy CI jest zielone (wszystkie 3 joby).

✓ **Weryfikacja:** PR ma zielone CI, zatwierdzenie recenzenta i uzupełniony szablon.

### Krok 4. Scalanie i wydanie

```
Scalanie przez GitHub (nie przez terminal!):
```

```
1. Kliknij 'Merge pull request' (lub 'Squash and merge' dla czystej historii)
Squash and merge: łączy wszystkie commity z brancha w jeden przed scaleniem.
Wynik: main ma jeden czysty commit zamiast 3 małych commitów z brancha.

2. Po scaleniu: sprawdź ze Issue #12 zostało automatycznie zamknięte.
(dzięki 'Closes #12' w opisie PR)

3. GitHub Actions release.yml uruchomi się automatycznie po push do main.
Ponieważ commit to 'feat:', semantic-release podniesie wersję minor:
np. 1.2.0 -> 1.3.0

4. Sprawdź GitHub > Releases - powinien pojawić się nowy release z:
- Tagiem (np. v1.3.0)
- Automatycznie wygenerowanym CHANGELOG
- Dołączonym APK (jeśli skonfigurowałeś w .releaserc.json)

5. Pobierz z galezi main najnowsze zmiany lokalnie:
git checkout main
git pull origin main
git tag # Sprawdź ze nowy tag jest widoczny
```

**Krok 1:** Scal PR przez GitHub ('Squash and merge').

**Krok 2:** Sprawdź ze Issue #12 jest zamknięte.

**Krok 3:** Poczekać na workflow release (1-2 minuty) i sprawdź GitHub Releases.

#### ✓ Weryfikacja

Issue zamknięte. PR scalony. Nowy tag i GitHub Release są widoczne. CHANGELOG.md zaktualizowany. Sprint zakończony.

## 24. Dodatek A. Cheat sheet: najważniejsze komendy i wzorce

...

### Git: komendy dnia codziennego

Komenda	Co robi
git status	Stan roboczego drzewa: co zmieniono, co jest staged, co jest nie-prześledzone.
git log --oneline --graph --all	Zwięzła historia commitów z grafem gałęzi. Najlepszy widok na stan repozytorium.
git diff HEAD	Różnica między bieżącym stanem a ostatnim commitem.
git stash / git stash pop	Odkładanie zmian 'na bok' bez commitowania. Użyteczne przy przełączaniu gałęzi.
git commit --amend --no-edit	Dodaj staged zmiany do ostatniego commitu bez zmiany opisu. TYLKO lokalnie!

<code>git rebase -i HEAD~3</code>	Interaktywny rebase ostatnich 3 commitów: połącz, zmień kolejność, edytuj opisy.
<code>git bisect start / good / bad</code>	Binarne wyszukiwanie commitu, który wprowadził błąd. Nieocenione przy regresji.
<code>git cherry-pick &lt;sha&gt;</code>	Przenieś wybrany commit z innej gałęzi bez mergowania całej gałęzi.
<code>git push --force-with-lease</code>	Bezpieczny force push: nie nadpisze zmian wypchniętych przez innego.
<code>git tag v1.0.0 &amp;&amp; git push --tags</code>	Ręczne stworzenie i wyłączanie tagu semver.

## Gradle - komendy testowania

Komenda	Co uruchamia
<code>./gradlew test</code>	Wszystkie testy jednostkowe (JVM, bez emulatora).
<code>./gradlew testDebugUnitTest</code>	Testy jednostkowe wariantu Debug.
<code>./gradlew test --tests '*FooTest*'</code>	Konkretna klasa testowa (wildcard).
<code>./gradlew connectedAndroidTest</code>	Testy instrumentowane (Espresso) - wymaga podłączonego emulatora.
<code>./gradlew jacocoTestReport</code>	Raport pokrycia kodu (po uruchomieniu testów).
<code>./gradlew detekt</code>	Analiza statyczna Detekt.
<code>./gradlew ktlintCheck</code>	Weryfikacja formatowania kodu.
<code>./gradlew ktlintFormat</code>	Automatyczna naprawa formatowania.
<code>./gradlew recordPaparazziDebug</code>	Aktualizacja golden images Paparazzi.
<code>./gradlew verifyPaparazziDebug</code>	Weryfikacja snapshot testów (używaj w CI).
<code>./gradlew :macrobenchmark:connectedBenchmarkAndroidTest</code>	Benchmarki Macrobenchmark.
<code>./gradlew dependencies   grep 'FAILED'</code>	Znajdź problemy z rozwiązywaniem zależności.

## MockK - najważniejsze funkcje

Funkcja MockK	Opis
<code>mockk&lt;T&gt;()</code>	Tworzy mock interfejsu lub klasy (open).
<code>mockk&lt;T&gt;(relaxed = true)</code>	Mock zwracający wartości domyślne - bez jawnych coEvery.
<code>spyk(realObject)</code>	Spy na prawdziwym obiekcie, metody można selektywnie podmieniać.
<code>every { obj.method() } returns value</code>	Definicja zachowania dla zwykłej (nie-suspend) metody.
<code>coEvery { obj.suspend() } returns value</code>	Definicja zachowania dla funkcji suspend (coroutine).
<code>coEvery { } answers { firstArg&lt;T&gt;() }</code>	Odpowiedź zależna od argumentu wywołania.
<code>coEvery { obj.method(capture(slot)) } just Runs</code>	Przechwytywanie argumentu do slot.
<code>verify(exactly = 1) { obj.method() }</code>	Weryfikacja liczby wywołań (zwykła metoda).
<code>coVerify { obj.suspend() }</code>	Weryfikacja wywołania funkcji suspend.

coVerifyOrder { a(); b() }	Weryfikacja kolejności wywołań
slot.captured	Dostęp do przechwyconego argumentu.
any(), eq("val"), neq("val")	Matchers argumentów: dowolny, równy, nierówny.

## Conventional Commits - szybka referencja

Typ	Kiedy używać	Wpływ na wersję (semver)	Przykład
feat:	Nowa funkcja widziana przez użytkownika	MINOR: 1.0 -> 1.1	feat(search): dodaj autouzupełnianie
fix:	Naprawa błędu widzianego przez użytkownika	PATCH: 1.0.0 -> 1.0.1	fix(api): popraw parsowanie -15 stopni
perf:	Optymalizacja wydajności	PATCH	perf(list): zastap FlatList FlashList
refactor:	Zmiany bez zmiany zachowania	Brak wydania	refactor: wyodrębnij WeatherMapper
test:	Dodanie lub naprawa testów	Brak wydania	test: dodaj testy ViewModelu
docs:	Dokumentacja (README, KDoc, komentarze)	Brak wydania	docs: dodaj opis API w README
ci:	Zmiany w plikach CI/CD	Brak wydania	ci: dodaj cache Gradle w GitHub Actions
chore:	Inne: aktualizacja zależności, konfiguracja narzędziowa	Brak wydania	chore(deps): aktualizuj MockK do 1.14
feat!:	Breaking change (wykrzyknik po typie)	MAJOR: 1.0 -> 2.0	feat!: zmień format API odpowiedzi

## Dodatek B. Najczęstsze błędy podczas ćwiczeń i ich rozwiązania

...

Błąd / objaw	Przyczyna i rozwiązanie krok po kroku
MockK: 'io.mockk.MockKException: no answer found for WeatherRepository.getWeather(...)'	Wywołano metodę mocka bez uprzedniego coEvery. Każda metoda mocka (nie-relaxed) musi mieć definicję zachowania. Dodaj: coEvery { repository.getWeather(any()) } returns Result.success(fakeData). Alternatywnie: użyj mockk(relaxed = true) jeśli nie potrzebujesz weryfikacji.
runTest nie kończy się - test wisi w nieskończoność	Coroutine czeka na zdarzenie które nigdy nie nastąpi: awaitItem() gdy Flow nie emituje, advanceUntilIdle() gdy jest nieskończona petla. Sprawdź czy: (1) Flow jest StateFlow (emituje stan początkowy), (2) nie ma nieskończonej petli w testowanej logice, (3) nie brakuje advanceUntilIdle() po akcji.
Turbine: 'No items were received but expected one'	Flow nie wyemitował wartości przed awaitItem(). Najczęściej: działanie (np. addFavorite) zostało wywołane PO bloku test { }. Przenies akcję WEWNATRZ bloku .test { }. Sprawdź że Flow jest 'hot' (StateFlow emituje natychmiast po subskrypcji).
Paparazzi: 'Golden file not found' lub 'Files differ'	Brak golden image (pierwsze uruchomienie) lub wygląd się zmienił. Dla pierwszego uruchomienia: ./gradlew recordPaparazziDebug. Dla celowej zmiany wyglądu: tak samo. Sprawdź, czy src/test/snapshots/ jest committed do Git.
GitHub Actions: 'Resource not ac-	Workflow nie ma uprawnień do wykonania akcji (np. tworzenie Release).

cessible by integration'   Semantic-release: 'The release 1.0.0 is already defined'	Dodaj do pliku .yml sekcje permissions: z odpowiednimi prawami: contents: write dla release, security-events: write dla CodeQL.   Tag już istnieje w repozytorium. Semantic-release nie nadpisze istniejącego tagu. Skasuj tag ręcznie: <code>git tag -d v1.0.0 &amp;&amp; git push origin :refs/tags/v1.0.0</code>. Albo zacznij od wyższej wersji przez .releaserc.json 'tagFormat'.
Room test: 'Cannot access database on the main thread'   Dependabot PR nie kompiluje się po aktualizacji zależności	Test Room wywołuje operacje bazy na głównym wątku bez <code>allowMainThreadQueries()</code>. Dodaj: <code>.allowMainThreadQueries()</code> do <code>Room.inMemoryDatabaseBuilder</code>. UWAGA: ta opcja jest TYLKO do testów - nigdy nie używaj w kodzie produkcyjnym.   Aktualizacja biblioteki zmieniła API (breaking change). Przejdź do pliku PR Dependabota, sprawdź CHANGELOG zaktualizowanej biblioteki. Dostosuj kod do nowego API. Merguj PR dopiero gdy CI jest zielone.
Branch Protection: nie można scalic PR ('Required status check has not passed')	Job CI z nazwą podaną w Branch Protection Rules nie przeszedł lub nie został uruchomiony. Sprawdź nazwę joba w ci.yml - musi być dokładnie taka sama jak w Branch Protection. Sprawdź logi jobu pod kątem błędów.
CodeQL: 'Extraction failed: java' przy budowaniu	CodeQL nie może skompilować projektu podczas analizy. Sprawdź, że krok <code>assembleDebug</code> w workflow <code>codeql.yml</code> przechodzi. CodeQL musi widzieć skompilowany kod. Dodaj <code>--continue</code> do <code>assembleDebug</code> lub przejdź na <code>autobuild</code>.